SKY TAKE OUT • ARCHITECTURE WHITEPAPER

服务端登录认证与令牌流转全链路架构概览

深入解析苍穹外卖管理端（Admin）与用户端（WeChat User）的双轨身份核验、JWT 令牌签发、拦截器鉴权闭环及 ThreadLocal 线程上下文透传机制。

所属工程	认证标准	鉴权核心	架构体系
sky-take-out	**JWT (HS256)**	**Spring Interceptor + ThreadLocal**	**管理端/用户端 双轨架构**

一、架构核心机制与双轨认证体系

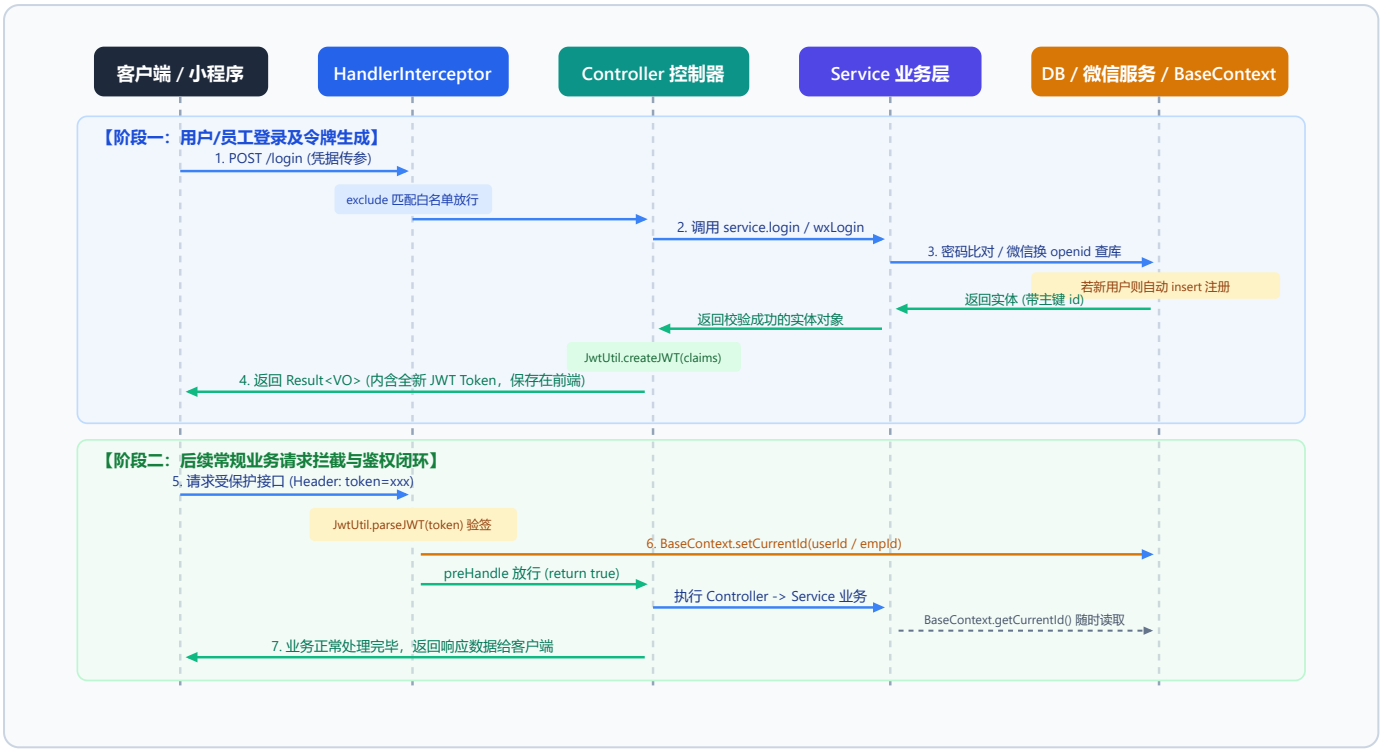
在苍穹外卖系统中，服务端采用基于 **JWT（JSON Web Token）的无状态会话机制**。服务端不维系集中式 Session，用户每一次身份认证成功后，由服务端使用配置密钥签发附带用户唯一主键（id）的加密字符串返回客户端，后续业务请求前端均通过 HTTP Header 携带令牌完成身份鉴定。

业务端	接口路径	身份凭证与认证源	自动注册与初次登录	令牌载荷 (Payload) 与拦截范围
管理端 (Admin)	/admin/employee/login	用户名 + MD5 加密密码（数据库预置员工数据）	**无自动注册** 所有账号由超管预先录入	empId -> Long 拦截 /admin/** 放行 /login
用户端 (User)	/user/user/login	微信临时凭证 code（调用微信官方服务换 openid）	**静默自动注册** 首次使用自动 insert 用户	userId -> Long 拦截 /user/** 放行 /login, /status

> **核心原则：**JWT 令牌的签发必须以**“凭证核验完全成功”**为绝对前置条件。任何账号不存在、密码比对失败、账号锁定、微信 code 失效，都会通过抛出异常由全局异常处理器阻断流程，绝不会下发令牌。

二、全链路交互时序图（SVG 架构全景）

下图完整呈现了从前端提交凭证、服务端逐层校验、签发 JWT，到后续发起受保护业务请求时的拦截鉴权全流程：


客户端 / 小程序
HandlerInterceptor
Controller 控制器
Service 业务层
DB / 微信服务 / BaseContext
【阶段一：用户/员工登录及令牌生成】
1. POST /login (凭据传参)
exclude 匹配白名单放行
2. 调用 service.login / wxLogin
3. 密码比对 / 微信换 openid 查库
若新用户则自动 insert 注册
返回实体 (带主键 id)
返回校验成功的实体对象
JwtUtil.createJWT(claims)
4. 返回 Result<VO> (内含全新 JWT Token，保存在前端)
【阶段二：后续常规业务请求拦截与鉴权闭环】
5. 请求受保护接口 (Header: token=xxx)
JwtUtil.parseJWT(token) 验签
6. BaseContext.setCurrentId(userId / empId)
preHandle 放行 (return true)
执行 Controller -> Service 业务
BaseContext.getCurrentId() 随时读取
7. 业务正常处理完毕，返回响应数据给客户端

三、管理端（员工）登录与令牌签发流程源码剖析

1. 控制器层接收与令牌封装

代码位于 EmployeeController.java。接收前端传递的账号密码，委托业务层校验通过后签发包含 empId 的 JWT。

EmployeeController.java — REST 接口

```
@PostMapping("/login") public Result<EmployeeLoginVO> login(@RequestBody EmployeeLoginDTO dto) { // 1. 业务层核心校验 Employee emp = employeeService.login(dto); // 2. 签发 JWT 令牌 Map<String, Object> claims = new HashMap<>(); claims.put(JwtClaimsConstant.EMP_ID, emp.getId()); String token = JwtUtil.createJWT( jwtProperties.getAdminSecretKey(), jwtProperties.getAdminTtl(), claims ); // 3. 组装脱敏 VO（剔除密码） EmployeeLoginVO vo = EmployeeLoginVO.builder() .id(emp.getId()).userName(emp.getUsername()) .name(emp.getName()).token(token).build(); return Result.success(vo); }
```

2. 业务层三级防线校验

代码位于 EmployeeServiceImpl.java。通过三道关卡确保仅有合法、激活的用户能进入签发令牌逻辑。

EmployeeServiceImpl.java — 业务校验

```
public Employee login(EmployeeLoginDTO dto) { // 关卡 1：用户名存在性核验 Employee emp = employeeMapper.getByUsername(dto.getUsername()); if (emp == null) { throw new AccountNotFoundException(...); } // 关卡 2：密码 MD5 加密比对 String md5Pwd = DigestUtils.md5DigestAsHex( dto.getPassword().getBytes() ); if (!md5Pwd.equals(emp.getPassword())) { throw new PasswordErrorException(...); } // 关卡 3：账号启用状态检测（1-启用 0-锁定） if (emp.getStatus() == StatusConstant.DISABLE) { throw new AccountLockedException(...); } return emp; }
```

四、用户端（微信小程序）登录与免密自动注册

用户端小程序无需用户输入账号密码，前端调用微信原生 wx.login() 换得授权码 code，服务端向微信官方服务器校验并完成用户的"识别/自动注册"一体化：

1. **接收临时凭证 code 并向微信验证**
   调用 https://api.weixin.qq.com/sns/jscode2session 接口，传入服务端的 appid、secret 和前端的 code，换取当前微信用户的全局唯一标识 openid。

2. **数据库 openid 查询与静默自动注册（核心亮点）**
   通过 userMapper.getByOpenid(openId) 查询。如果为 null，立即构建 User.builder().openid(openId).createTime(now).build() 并执行插入。借助 MyBatis 的 useGeneratedKeys="true"，主键自增 ID 自动回填到 user.id。

3. **签发小程序专属 JWT 令牌**
   将用户自增主键注入载荷 claims.put(JwtClaimsConstant.USER_ID, user.getId())，通过 JwtUtil.createJWT(...) 使用用户端专属密钥生成 Token 返回给小程序。

五、拦截器鉴权闭环与 ThreadLocal 机制

获取 Token 之后，客户端在后续请求头（token）中携带该字符串。服务端通过 Spring MVC 拦截器实现无状态拦截鉴权：

拦截器执行逻辑 (JwtTokenAdminInterceptor)

1. **排除非 Controller 资源**：若拦截到的不是 `HandlerMethod`（如静态资源），直接放行。
2. **请求头提取 Token**：`request.getHeader("token")`。
3. **JwtUtil 验签解密**：调用 `Jwts.parser().setSigningKey(...).parseClaimsJws(token)`。若令牌过期、被伪造或篡改，抛出异常。
4. **失败中断**：进入 catch 块，设置 `response.setStatus(401)` 并返回 `false`。
5. **绑定线程上下文**：提取 `empId / userId`，存入 `BaseContext.setCurrentId(id)`。

BaseContext 与 ThreadLocal 深度透视

Tomcat 在接收 HTTP 请求时，会从工作线程池分配一个专门的线程来处理当前请求，整个过程处于**同一个线程生命周期**中：

BaseContext.java ThreadLocal 封装

```
public class BaseContext { public static ThreadLocal<Long> threadLocal = new ThreadLocal<>(); public static void setCurrentId(Long id) { threadLocal.set(id); } public static Long getCurrentId() { return threadLocal.get(); } public static void removeCurrentId() { threadLocal.remove(); } }
```

六、关键业务问答与知识卡片总结

高频关键问题	精准答案与原理总结
1. 这个登录属于第一次登录吗?	• **管理端（员工）**：不属于第一次登录概念。员工账号由管理员在后台提前新增录入，每次登录都在验证已有数据，验证通过后都会重新生成全新 JWT。 • **用户端（微信）**：包含新用户首次登录逻辑。若数据库无此 openid，系统会自动 insert 注册并生成新用户，老用户则直接查出。但无论新老，只要校验成功都会签发全新 JWT。
2. 什么时候生成令牌？什么时候只解密校验?	• **生成令牌**：仅在前端显式调用 `/login` 接口，且后端业务层**对密码/微信凭据校验完全通过后**生成一次。 • **只解密校验**：后续的所有普通业务请求（如菜品分页查询、下单、修改状态等），拦截器只调用 `parseJWT` 验签提取 ID，**绝不重复生成新令牌。**
3. 服务端是如何实现退出登录（Logout）的?	由于 JWT 是完全无状态的，服务端内存与 DB 中不维系 Token 会话。后端的 `/logout` 只需直接返回 `Result.success()`，真正的退出由**前端主动删除本地缓存中的 Token** 即可达成。
4. ThreadLocal 的线程安全与复用防坑	Tomcat 采用线程池复用机制，处理完请求后该线程不会销毁而是回到池中。在生产最佳实践中，应在拦截器的 `afterCompletion` 中调用 `BaseContext.removeCurrentId()`，防止脏数据串染与内存泄漏隐患。